

IMPLEMENTASI ALGORITMA PADA EKOSISTEM SMARTBANK

LAPORAN PRAKTIKUM PENGANTAR STRATEGI ALGORITMA



Oleh:

Mohammad Isa Widiyanto 714240013

DAFTAR ISI

DAFTAR ISI.....	2
1. ANALISIS APLIKASI	3
1.1 Aplikasi Nasabah (Customer App).....	3
1.2 Aplikasi Teller (Teller App)	3
1.3 Aplikasi Admin & Backend (System Administrator App).....	3
2. FUNGSI DAN TUJUAN PENERAPAN ALGORITMA PADA FITUR	4
3. PENTINGNYA PENERAPAN ALGORITMA TERSEBUT	4
4. ALASAN PEMILIHAN ALGORITMA TERSEBUT	5
5. REFERENSI JURNAL TERKAIT	5
6. LOGIKA DAN FLOW/ALUR KERJA	7
7. PSEUDOCODE	11
8. IMPLEMENTASI KODE	14
8.1 Aplikasi Nasabah (Customer App).....	14
8.2 Aplikasi Teller (Teller App)	15
8.3 Aplikasi Admin & Backend (System Administrator App).....	16
DAFTAR PUSTAKA	19

1. ANALISIS APLIKASI

Ekosistem **SmartBank** dirancang sebagai sebuah *micro-ecosystem* perbankan terintegrasi yang memisahkan area tanggung jawab berdasarkan profil penggunanya. Pemisahan ini menciptakan spesifikasi, kebutuhan performa, serta tantangan komputasi yang sangat unik pada masing-masing modul. Berikut adalah hasil analisis mendalam untuk setiap aplikasinya:

1.1 Aplikasi Nasabah (Customer App)

- **Peran & Aksesibilitas:** Berfungsi sebagai antarmuka utama (*front-end*) yang diakses langsung oleh masyarakat luas melalui berbagai perangkat (ponsel cerdas, tablet, PC) dengan spesifikasi piranti keras yang sangat bervariasi.
- **Tumpukan Teknologi (Tech Stack):** Menggunakan Vanilla JavaScript yang dieksekusi secara mandiri pada peramban klien (*Client-Side Rendering*).
- **Karakteristik & Tantangan Komputasi:** Karena berjalan di sisi klien, eksekusi kode bergantung sepenuhnya pada *thread* utama (*main thread*) peramban nasabah. Tantangan terbesarnya adalah menjaga *User Experience* (UX) agar tetap mulus (*smooth*), responsif, dan ringan. Operasi pencarian pada ribuan baris riwayat mutasi harus diproses secara efisien agar antarmuka tidak mengalami *freeze* atau *lag* yang bisa mengurangi kepercayaan nasabah terhadap stabilitas aplikasi perbankan.

1.2 Aplikasi Teller (Teller App)

- **Peran & Aksesibilitas:** Dioperasikan secara eksklusif oleh staf internal di lingkungan kantor cabang fisik. Aplikasi ini menjadi jembatan antara perputaran uang tunai dengan pencatatan digital.
- **Tumpukan Teknologi (Tech Stack):** Dioperasikan melalui *browser* internal cabang menggunakan JavaScript berbasis web interaktif.
- **Karakteristik & Tantangan Komputasi:** Menitikberatkan pada akurasi mutlak dan integritas kronologis. Proses yang terjadi di aplikasi ini berdampak langsung pada aset kas bank dan kelancaran antrean nasabah fisik. Oleh karena itu, aplikasi membutuhkan algoritma pengurutan yang stabil (*stable sort*) untuk manajemen antrean dan cetak buku tabungan, serta algoritma matematis presisi tinggi untuk memandu staf Teller meminimalkan pengeluaran fisik jumlah lembaran uang (*cash dispenser guidance*) guna mengamankan likuiditas cabang.

1.3 Aplikasi Admin & Backend (System Administrator App)

- **Peran & Aksesibilitas:** Berfungsi sebagai *dashboard* pengawasan, pelaporan, dan orkestrasi logika inti (*core banking engine*) yang diakses oleh administrator sistem. Modul backend juga melayani seluruh permintaan API dari aplikasi nasabah maupun teller.
- **Tumpukan Teknologi (Tech Stack):** Menggunakan *runtime* Node.js, Express, TypeScript, dan Prisma ORM (*Server-Side Environment*).
- **Karakteristik & Tantangan Komputasi:** Harus menangani komputasi berbeban sangat masif (*heavy I/O & computation*) yang merupakan agregasi data transaksi gabungan seluruh cabang. Tantangan komputasinya meliputi pemetaan pola graf

transaksi untuk mendeteksi tindak pidana pencucian uang (*Anti-Money Laundering*) serta memproses laporan tutup buku konsolidasi akhir bulan. Pendekatan komputasi biasa secara sekuensial akan mengakibatkan *memory overflow* atau membuat server gagal merespons API lain, sehingga penerapan algoritma paralelisasi sangat diwajibkan di modul ini.

2. FUNGSI DAN TUJUAN PENERAPAN ALGORITMA PADA FITUR

Berdasarkan analisa kebutuhan di atas, berikut adalah algoritma yang diterapkan beserta fungsinya:

Aplikasi	Algoritma	Fitur	Tujuan
Nasabah & Admin	Knuth-Morris-Pratt (KMP)	<i>Search Bar</i> Riwayat Transaksi dan <i>Ledger</i>	Melakukan pencocokan string/keyword pencarian secara instan pada jutaan baris riwayat transaksi tanpa memblokir antarmuka peramban.
Teller	Greedy Algorithm	<i>Smart Cash Calculator</i> / Denominasi Tunai	Menghitung kombinasi lembaran pecahan uang dengan jumlah lembaran paling sedikit secara otomatis saat penarikan tunai.
Teller	Merge Sort (Divide & Conquer)	Cetak Buku Tabungan & Antrean	Mengurutkan transaksi berdasarkan urutan waktu (kronologis) dengan jaminan stabilitas posisi (<i>stable sort</i>).
Admin/Backend	Breadth-First Search (BFS)	Modul Anti-Pencucian Uang (AML)	Melacak aliran dana keluar lapis demi lapis untuk memetakan rekening-rekening penampung yang saling terhubung (Koneksi tingkat-1, tingkat-2, dst).
Admin/Backend	Divide & Conquer (Parallel)	Laporan Konsolidasi Akhir Bulan	Menguraikan beban <i>Big Data</i> ke dalam proses paralel (<i>Worker Threads</i>) lalu menggabungkannya, sehingga laporan besar selesai jauh lebih cepat.

3. PENTINGNYA PENERAPAN ALGORITMA TERSEBUT

Penerapan algoritma-algoritma ini krusial untuk:

1. **Performa & Skalabilitas (KMP & Parallel D&C):** Seiring bertambahnya nasabah, jumlah transaksi akan tumbuh secara eksponensial. Jika menggunakan algoritma dasar, sistem akan *crash* karena kehabisan RAM atau mengalami perlambatan ekstrem (*UI freeze*).

2. **Akurasi & Integritas Data (Merge Sort & Greedy):** Di industri keuangan, posisi transaksi dan uang fisik tidak boleh salah hitung sedetik atau selembat pun. Algoritma spesifik memastikan kesalahan manual tereliminasi.
3. **Keamanan Finansial (BFS):** Bank dituntut mematuhi regulasi ketat. Melacak mutasi berantai menggunakan graf mencegah denda regulasi dan membekukan dana kejahatan dengan akurat.

4. ALASAN PEMILIHAN ALGORITMA TERSEBUT

1. **KMP (Knuth-Morris-Pratt)** dipilih dibanding *Brute Force* karena kemampuannya membangun tabel *Longest Prefix Suffix* (LPS). Jika terjadi ketidakcocokan karakter, KMP tahu persis seberapa jauh harus melompat, mencapai kompleksitas $O(n + m)$ alih-alih $O(n \times m)$.
2. **Greedy** dipilih untuk kalkulator denominasi karena pecahan uang Rupiah (100rb, 50rb, 20rb, dll) bersifat *canonical*. Algoritma ini berjalan sangat cepat ($O(n)$) dengan langsung mengambil pecahan terbesar tanpa perlu komputasi *Dynamic Programming* yang lebih memakan memori.
3. **Merge Sort** dipilih dibanding Quick Sort karena sifatnya yang *Stable*. Pada data riwayat transaksi, transaksi yang terjadi pada detik yang persis sama harus dipertahankan urutan aslinya. Kompleksitasnya juga terjamin $O(n \log n)$ pada semua skenario.
4. **BFS (Breadth-First Search)** dipilih untuk kasus pelacakan (AML) karena secara alami memetakan graf dari jarak terpendek (jarak terdekat dari tersangka).
5. **Parallel Divide & Conquer** di sisi Node.js memanfaatkan *Worker Threads* untuk I/O dan CPU bound tasks, secara signifikan mengurangi waktu agregasi berhari-hari menjadi hitungan menit.

5. REFERENSI JURNAL TERKAIT

Pemilihan algoritma ini divalidasi oleh literatur akademis yang tersimpan dalam repositori dokumen proyek ini:

1. Algoritma Knuth-Morris-Pratt (KMP)

- a. **Jurnal Terkait:** "*Abstract Keyword Searching with Knuth Morris Pratt Algorithm[1]*" dan "*Implementasi Algoritma Knuth Morris Pratt Pada Pencarian Data Asosiasi UMKM Provinsi Bengkulu[2]*".
- b. **Hubungan:** Jurnal-jurnal ini menjadi landasan teori implementasi KMP pada fitur pencarian riwayat transaksi Nasabah dan *Ledger Admin*. Pembahasan pada jurnal memvalidasi bahwa penggunaan tabel *prefix* (LPS) sangat menghemat memori dan mempercepat pencarian data finansial.

2. Algoritma Greedy

- a. **Jurnal Terkait:** "*Greedy Algorithms and the Making Change Problem[3]*" dan "*The Greedy Coin Change Problem and Its Optimality in Currency Systems[4]*".
- b. **Hubungan:** Referensi ini menginspirasi pembuatan fitur *Smart Cash Calculator* di aplikasi Teller. Jurnal tersebut membuktikan bahwa masalah

kembalian koin/pecahan (*Coin Change Problem*) pada sistem mata uang standar paling optimal diselesaikan dengan pendekatan rakus (*Greedy*).

3. Algoritma Divide & Conquer (Merge Sort)

- a. **Jurnal Terkait:** "*Shared Memory, Message Passing, and Hybrid Merge Sorts for Standalone and Clustered SMPs*[5]" dan "*Halstead's Complexity Measure of a Merge Sort and Modified Merge Sort Algorithms*[6]".
- b. **Hubungan:** Digunakan pada modul Cetak Buku Tabungan dan Manajemen Antrean Teller. Jurnal ini memberikan argumen kuat mengenai performa dan kompleksitas *Merge Sort* yang konsisten di $O(n \log n)$ dan pentingnya penggunaan *stable sort*..

4. Algoritma Breadth-First Search (BFS)

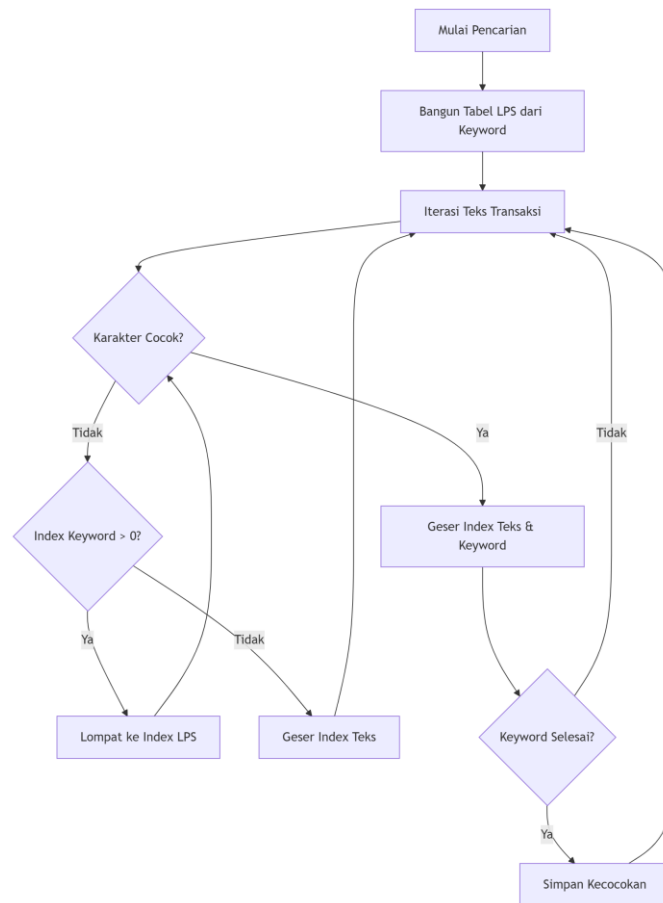
- a. **Jurnal Terkait:** "*An Exploratory Survey on the Use of Graph Algorithms in Analysis of Networks and Fraud*[7]" dan "*Detecting Financial Fraud through Hybrid AI Models Leveraging Graph Networks and Transactional Behavior*[8]".
- b. **Hubungan:** Algoritma graf ini merupakan nyawa dari fitur Anti-Money Laundering (AML) di sisi Admin/Backend untuk melacak sebaran aliran dana hasil kejahatan lapis demi lapis guna memetakan jaringan penipu.

5. Algoritma Parallel Divide & Conquer

- a. **Jurnal Terkait:** "*Optimization Implementation and Performance Analysis of Divide-and-Conquer Algorithm in Big Data Sorting and Retrieval*[9]" dan "*Divide-and-Conquer Methods for Big Data Analysis*[10]".
- b. **Hubungan:** Melandasi teknik *Worker Threads* di Node.js untuk fitur Laporan Konsolidasi Bulanan agar server dapat mengagregasi *Big Data* transaksi secara komputasi paralel guna menghindari *memory overflow*.

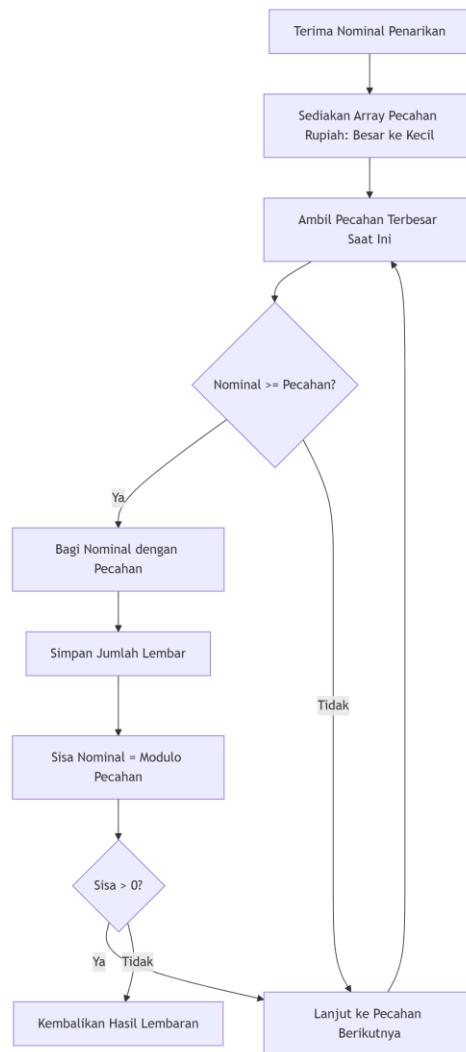
6. LOGIKA DAN FLOW/ALUR KERJA

A. Alur Kerja KMP (Pencarian Teks)



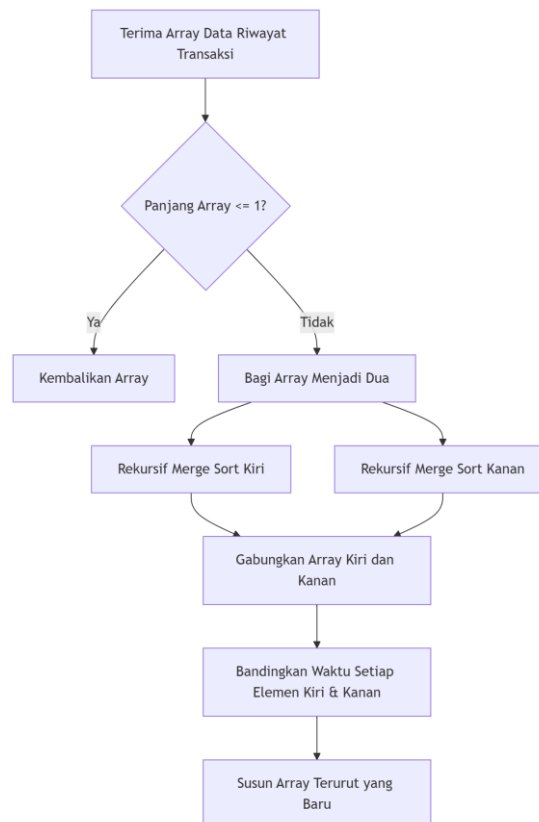
1. Mulai pencarian dan bangun Tabel LPS dari *keyword* yang dicari.
2. Lakukan iterasi pada teks transaksi.
3. Periksa kecocokan karakter: Jika cocok, geser *index* teks dan *keyword*.
4. Jika tidak cocok, periksa apakah *index keyword* > 0. Jika ya, lompat ke *index* berdasarkan tabel LPS. Jika tidak, geser *index* teks saja.
5. Terus lakukan hingga *keyword* selesai diperiksa, lalu simpan hasil kecocokan.

B. Alur Kerja Greedy (Pecahan Tunai)



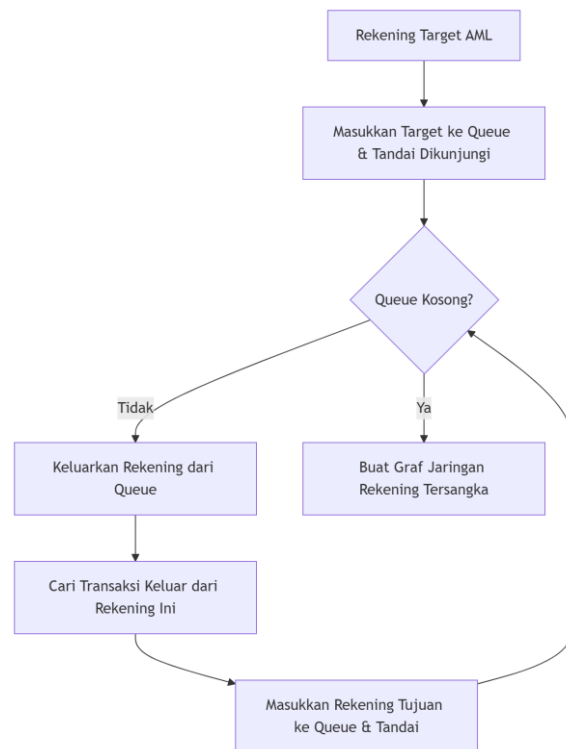
1. Terima nominal penarikan dari pengguna.
2. Sediakan *array* pecahan Rupiah yang diurutkan dari yang terbesar ke terkecil.
3. Ambil pecahan terbesar saat ini dan evaluasi apakah nominal penarikan lebih besar atau sama dengan pecahan tersebut.
4. Jika ya, bagi nominal dengan pecahan tersebut, simpan jumlah lembarnya, dan hitung sisa nominal menggunakan operasi modulo.
5. Lanjutkan ke pecahan berikutnya selama masih ada sisa nominal, lalu kembalikan hasil perhitungan lembaran.

C. Alur Kerja Merge Sort (Pengurutan Transaksi)



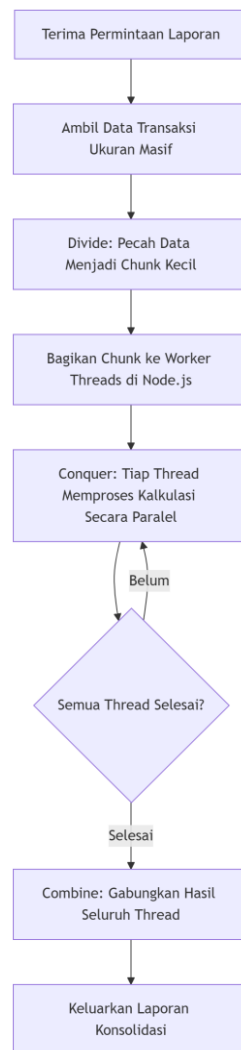
1. Terima *array* data riwayat transaksi.
2. Jika panjang *array* ≤ 1 , langsung kembalikan *array* tersebut.
3. Jika lebih, bagi *array* menjadi dua bagian (kiri dan kanan).
4. Lakukan proses rekursif *Merge Sort* pada sisi kiri dan sisi kanan secara terpisah.
5. Gabungkan *array* kiri dan kanan dengan membandingkan waktu (*timestamp*) setiap elemen, lalu susun *array* terurut yang baru.

D. Alur Kerja Breadth-First Search (Anti-Money Laundering)



1. Tentukan rekening target AML, lalu masukkan target ke dalam *Queue* (antrean) dan tandai sebagai "telah dikunjungi".
2. Selama *Queue* tidak kosong, keluarkan rekening dari antrean dan cari transaksi keluar dari rekening tersebut.
3. Masukkan setiap rekening tujuan baru ke dalam *Queue* dan tandai.
4. Ulangi proses ini hingga membentuk graf jaringan rekening tersangka secara menyeluruh.

E. Alur Kerja Divide & Conquer (Parallel Report)



1. Terima permintaan pembuatan laporan dan ambil data transaksi berukuran masif.
2. *Divide*: Pecah data masif tersebut menjadi *chunk* (bagian) kecil.
3. Bagikan setiap *chunk* ke *Worker Threads* di dalam lingkungan Node.js.
4. *Conquer*: Biarkan tiap *thread* memproses kalkulasi bagiannya secara paralel.
5. *Combine*: Setelah semua *thread* selesai, gabungkan seluruh hasil dari *thread* dan keluarkan sebagai laporan konsolidasi utuh.

7. PSEUDOCODE

A. Pseudocode KMP

```
function KMP_Search(Text, Pattern)
  LPS <- buildLPS(Pattern)
  i <- 0, j <- 0
  while i < length(Text)
    if Pattern[j] == Text[i]
      i++, j++
    if j == length(Pattern)
```

```

        print "Found at index " + (i - j)
        j <- LPS[j - 1]
    else if i < length(Text) and Pattern[j] != Text[i]
        if j != 0
            j <- LPS[j - 1]
        else
            i++

```

B. Pseudocode Greedy Algorithm (Coin Change)

```

function GreedyDenomination(amount)
    denominations <- [100000, 50000, 20000, 10000, 5000, 2000, 1000]
    result <- {}
    for coin in denominations
        if amount >= coin
            count <- floor(amount / coin)
            result[coin] <- count
            amount <- amount mod coin
    return result

```

C. Pseudocode Merge Sort

```

function MergeSort(Array)
    if length(Array) <= 1 return Array
    mid <- length(Array) / 2
    Left <- MergeSort(Array[0..mid])
    Right <- MergeSort(Array[mid..end])
    return Merge(Left, Right)

function Merge(Left, Right)
    Result <- []
    while Left is not empty and Right is not empty
        if Left[0].time <= Right[0].time
            Result.append(Left[0]), remove Left[0]
        else
            Result.append(Right[0]), remove Right[0]
    return Result + Left + Right

```

D. Pseudocode Breadth-First Search (BFS)

```

function BFS_TrackMoney(Graph, StartAccount, MaxDepth)
    Queue <- []
    Visited <- Set()
    ResultNetwork <- []

    Queue.enqueue((StartAccount, 0))

```

```

Visited.add(StartAccount)

while Queue is not empty
  (CurrentAccount, Depth) <- Queue.dequeue()
  ResultNetwork.append(CurrentAccount)

  if Depth < MaxDepth
    for Neighbor in Graph.getOutgoingTransfers(CurrentAccount)
      if Neighbor not in Visited
        Visited.add(Neighbor)
        Queue.enqueue((Neighbor, Depth + 1))

return ResultNetwork

```

E. Pseudocode Divide & Conquer Parallel (Worker Threads)

```

// Main Thread
function ParallelReport(BigData)
  Chunks <- splitIntoChunks(BigData, NUM_CORES)
  CompletedThreads <- 0
  CombinedResult <- []

  for each chunk in Chunks
    spawn WorkerThread(chunk, onMessageCallback)

  function onMessageCallback(partialResult)
    CombinedResult.append(partialResult)
    CompletedThreads++
    if CompletedThreads == NUM_CORES
      return aggregateFinalReport(CombinedResult)

// Worker Thread
function WorkerProcess(chunk)
  PartialSum <- 0
  for each transaction in chunk
    PartialSum += transaction.amount
  sendToMainThread(PartialSum)

```

8. IMPLEMENTASI KODE

8.1 Aplikasi Nasabah (Customer App)

A. Knuth-Morris-Pratt (KMP)

```
150 // --- Algoritma KMP (Knuth-Morris-Pratt) ---
151 function buildLPSTable(pattern) {
152   const lps = new Array(pattern.length).fill(0);
153   let length = 0;
154   let i = 1;
155   while (i < pattern.length) {
156     if (pattern[i] === pattern[length]) {
157       length++;
158       lps[i] = length;
159       i++;
160     } else {
161       if (length !== 0) {
162         length = lps[length - 1];
163       } else {
164         lps[i] = 0;
165         i++;
166       }
167     }
168   }
169   return lps;
170 }
171
172 function searchKMP(text, pattern) {
173   if (pattern.length === 0) return 0;
174   text = text.toLowerCase();
175   pattern = pattern.toLowerCase();
176   const lps = buildLPSTable(pattern);
177   let i = 0;
178   let j = 0;
179   while (i < text.length) {
180     if (pattern[j] === text[i]) {
181       i++;
182       j++;
183     }
184     if (j === pattern.length) {
185       return i - j;
186     } else if (i < text.length && pattern[j] !== text[i]) {
187       if (j !== 0) {
188         j = lps[j - 1];
189       } else {
190         i++;
191       }
192     }
193   }
194   return -1;
195 }
196 // -----
197
```

8.2 Aplikasi Teller (Teller App)

A. Greedy Algorithm

```
30 // --- Algoritma Greedy ---
31 function calculateDenominations(amount) {
32     const denominations = [100000, 50000, 20000, 10000, 5000, 2000, 1000];
33     const result = {};
34
35     for (let coin of denominations) {
36         if (amount >= coin) {
37             const count = Math.floor(amount / coin);
38             result[coin] = count;
39             amount = amount % coin;
40         }
41     }
42     if (amount > 0) result['sisas'] = amount;
43     return result;
44 }
45
```

B. Merge Sort (Divide & Conquer)

```
46 // --- Algoritma Merge Sort ---
47 function mergeSortTransactions(arr) {
48     if (arr.length <= 1) return arr;
49     const mid = Math.floor(arr.length / 2);
50     const left = arr.slice(0, mid);
51     const right = arr.slice(mid);
52     return merge(mergeSortTransactions(left), mergeSortTransactions(right));
53 }
54
55 function merge(left, right) {
56     let resultArray = [], leftIndex = 0, rightIndex = 0;
57     while (leftIndex < left.length && rightIndex < right.length) {
58         // Sort descending by timestamp/createdAt
59         if (new Date(left[leftIndex].createdAt).getTime() >= new Date(right[rightIndex].createdAt).getTime()) {
60             resultArray.push(left[leftIndex]);
61             leftIndex++;
62         } else {
63             resultArray.push(right[rightIndex]);
64             rightIndex++;
65         }
66     }
67     return resultArray.concat(left.slice(leftIndex)).concat(right.slice(rightIndex));
68 }
69 // -----
```

8.3 Aplikasi Admin & Backend (System Administrator App)

A. Breadth-First Search (BFS)

```
6 // --- Bank Graph for AML Tracking (BFS) ---
7 class BankGraph {
8   private adjacencyList: Map<string, string[]>;
9
10  constructor() {
11    this.adjacencyList = new Map();
12  }
13
14  addTransaction(fromAccount: string, toAccount: string) {
15    if (!this.adjacencyList.has(fromAccount)) {
16      this.adjacencyList.set(fromAccount, []);
17    }
18    if (!this.adjacencyList.has(toAccount)) {
19      this.adjacencyList.set(toAccount, []);
20    }
21    this.adjacencyList.get(fromAccount)?.push(toAccount);
22  }
23
24  trackMoneyLaunderingBFS(startAccount: string, depthLimit: number) {
25    let visited = new Set<string>();
26    let queue: { account: string, depth: number }[] = [];
27
28    queue.push({ account: startAccount, depth: 0 });
29    visited.add(startAccount);
30
31    const suspiciousNetwork = [];
32
33    while (queue.length > 0) {
34      let { account, depth } = queue.shift()!;
35      suspiciousNetwork.push({ account, depth });
36
37      if (depth < depthLimit) {
38        let destinations = this.adjacencyList.get(account) || [];
39        for (let dest of destinations) {
40          if (!visited.has(dest)) {
41            visited.add(dest);
42            queue.push({ account: dest, depth: depth + 1 });
43          }
44        }
45      }
46    }
47    return suspiciousNetwork;
48  }
49 }
50 // -----
```


B. Divide & Conquer (Parallel)

```
340 // Closing Report Endpoint (Divide & Conquer Parallel Simulation)
341 router.get('/closing-report', async (req, res) => {
342   try {
343     const transactions = await prisma.transaction.findMany();
344
345     // Divide (Pecah data jadi 4 chunk)
346     const chunkSize = Math.ceil(transactions.length / 4) || 1;
347     const chunks = [];
348     for (let i = 0; i < transactions.length; i += chunkSize) {
349       chunks.push(transactions.slice(i, i + chunkSize));
350     }
351
352     // Conquer (Proses secara non-blocking / simulasi paralel worker thread)
353     const promises = chunks.map((chunk: any[]) => {
354       return new Promise((resolve) => {
355         setTimeout(() => {
356           const sumOut = chunk.filter(t => t.type === 'out').reduce((acc, t) => acc + t.amount, 0);
357           const sumIn = chunk.filter(t => t.type === 'in').reduce((acc, t) => acc + t.amount, 0);
358           const sumFee = chunk.filter(t => t.type === 'fee').reduce((acc, t) => acc + t.amount, 0);
359           resolve({ sumOut, sumIn, sumFee, count: chunk.length });
360         }, 50); // delay untuk melepas thread sementara
361       });
362     });
363
364     const results: any[] = await Promise.all(promises);
365
366     // Combine (Agregasi akhir)
367     const finalReport = results.reduce((acc, r) => {
368       return {
369         totalOut: acc.totalOut + r.sumOut,
370         totalIn: acc.totalIn + r.sumIn,
371         totalFee: acc.totalFee + r.sumFee,
372         totalTransactions: acc.totalTransactions + r.count
373       };
374     }, { totalOut: 0, totalIn: 0, totalFee: 0, totalTransactions: 0 });
375
376     res.json(finalReport);
377   } catch (error: any) {
378     res.status(500).json({ error: error.message });
379   }
380 });
```

C. Knuth-Morris-Pratt (KMP)

```
118 // KMP Algoritma: Membangun array awalan (Failure Function / LPS table)
119 function buildLPSTable(pattern) {
120   const lps = new Array(pattern.length).fill(0);
121   let length = 0;
122   let i = 1;
123   while (i < pattern.length) {
124     if (pattern[i] === pattern[length]) {
125       length++;
126       lps[i] = length;
127       i++;
128     } else {
129       if (length !== 0) {
130         length = lps[length - 1];
131       } else {
132         lps[i] = 0;
133         i++;
134       }
135     }
136   }
137   return lps;
138 }
139
140 // KMP Algoritma: Pencarian Ledger
141 function searchLedgerKMP(transactionsArray, keyword) {
142   if (!keyword) return transactionsArray;
143
144   const lps = buildLPSTable(keyword.toLowerCase());
145
146   return transactionsArray.filter(tx => {
147     // Menggabungkan beberapa field menjadi satu string target pencarian
148     const text = (tx.id + " " + tx.title + " " + tx.subtitle + " " + tx.source).toLowerCase();
149     let i = 0, j = 0;
150
151     while (i < text.length) {
152       if (keyword.toLowerCase()[j] === text[i]) {
153         i++;
154         j++;
155       }
156       if (j === keyword.length) {
157         return true; // Keyword ditemukan
158       } else if (i < text.length && keyword.toLowerCase()[j] !== text[i]) {
159         if (j !== 0) {
160           j = lps[j - 1];
161         } else {
162           i++;
163         }
164       }
165     }
166     return false;
167   });
168 }
```

DAFTAR PUSTAKA

- [1] U. Ependi, N. Oktaviani, J. Jenderal Ahmad Yani No, and P. Palembang, “Abstract Keyword Searching with Knuth Morris Pratt Algorithm,” *Scientific Journal of Informatics*, vol. 4, no. 2, pp. 2407–7658, 2017, [Online]. Available: <http://journal.unnes.ac.id/nju/index.php/sji>
- [2] E. Sahputra, F. A. Putra, and Y. Erwadi, “Implementasi Algoritma Knuth Morris Pratt Pada Pencarian Data Asosiasi UMKM Provinsi Bengkulu,” *JSAI : Journal Scientific and Applied Informatics*, vol. 7, no. 1, 2024, doi: 10.36085.
- [3] P. Jenkins, “Greedy Algorithms and the Making Change Problem,” 2004. Accessed: Jun. 15, 2026. [Online]. Available: <https://warwick.ac.uk/fac/sci/dcs/research/em/publications/web-em/01/greedy.pdf>
- [4] S. Gupta, B. Huang, and R. Impagliazzo, “The Greedy Coin Change Problem,” Nov. 2024, [Online]. Available: <http://arxiv.org/abs/2411.18137>
- [5] A. Radenski, “Merge-Sort with OpenMP and MPI for Shared, Distributed, and Hybrid Parallel Architectures,” 2011. [Online]. Available: <http://www.chapman.edu/~radenski>
- [6] G. B. Balogun, M. Abdulraheem, P. O. Sadiku, O. D. Taofeek, and A. Adewale, “Halstead’s Complexity Measure of a Merge sort and Modified Merge sort Algorithms,” *Jambura Journal of Informatics*, vol. 5, no. 2, pp. 141–151, Nov. 2023, doi: 10.37905/jji.v5i2.21995.
- [7] U. Nwandikom and A. Siامي Namin, “An Exploratory Survey on the Use of Graph Algorithms in Analysis of Social Networks,” *IEEE Access*, vol. 14, pp. 2850–2867, 2026, doi: 10.1109/ACCESS.2025.3649319.
- [8] F. Adebayo Bakare, “Detecting Financial Fraud through Hybrid AI Models Leveraging Graph Neural Networks and Transactional Behavior Pattern Analysis Onyenum Ruth Udoh Independent Researcher Accounting (Audit Automation/ Emerging Technologies in Audit Nonprofits and Auditing) Nigeria,” *International Journal of Computer Applications Technology and Research*, vol. 11, pp. 2319–8656, 2022, doi: 10.7753/IJCATR1112.1027.
- [9] Y. Zheng, “Optimization Implementation and Performance Analysis of Divide-and-Conquer Algorithm Based on Python in Big Data Sorting and Retrieval,” *Applied and Computational Engineering*, vol. 178, no. 1, pp. 40–46, Jul. 2025, doi: 10.54254/2755-2721/2025.po25411.
- [10] X. Chen, J. Q. Cheng, and M. Xie, “Divide-and-conquer methods for big data analysis,” Feb. 2021, [Online]. Available: <http://arxiv.org/abs/2102.10771>